

Batch vs Stream Processing - What's the Difference?



ASHISH PRATAP SINGH

OCT 19, 2024



135



1



8

Share

In the world of big data, **batch processing** and **stream processing** are the two common approaches to **process large amounts of data**.

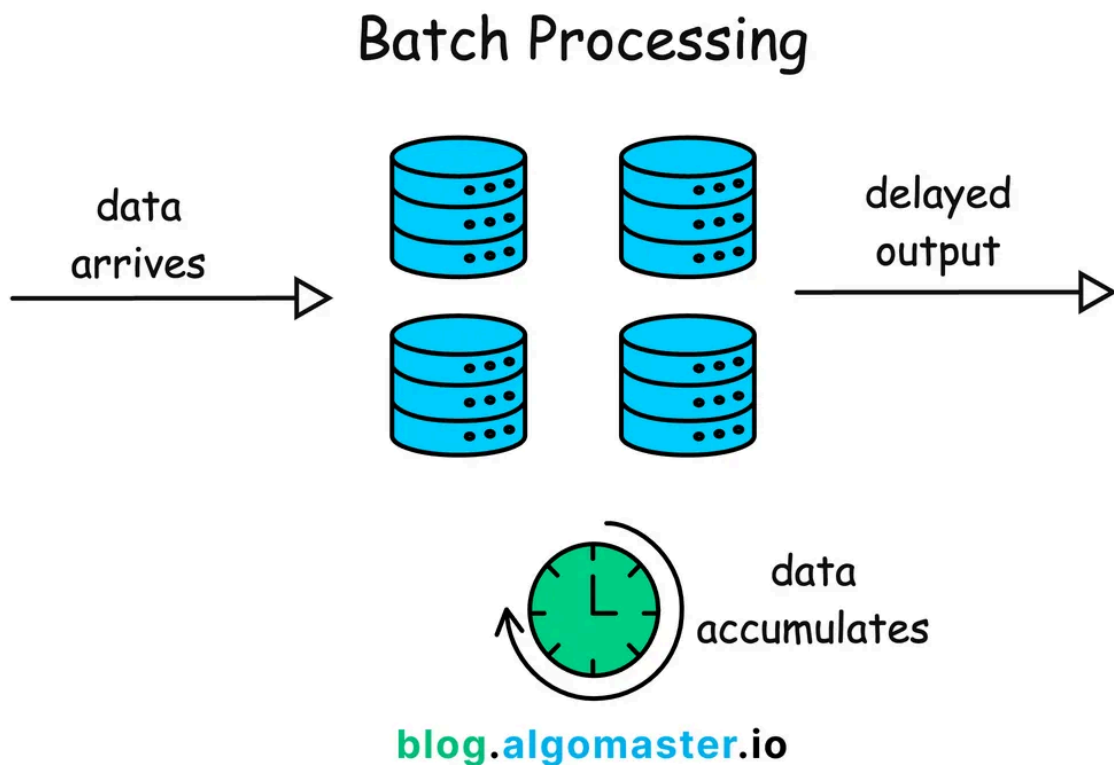
While both the approaches aim to process data, they differ in **execution** and are suited for different use cases.

In this article, we'll dive into the details of batch processing and stream processing, explore their characteristics, differences, and provide examples to clarify their use cases.

If you're enjoying this newsletter and want to get even more value, consider becoming a [paid subscriber](#).

As a paid subscriber, you'll unlock all **premium articles** and gain full access to all [premium courses](#) on [algomaster.io](#).

1. Batch Processing



Batch processing is a traditional approach to data processing that has been around decades.

It involves:

1. **Collecting and storing** data over a period of time (hours, days, or even weeks).
2. Processing this data in **bulk** at **scheduled intervals**.
3. Producing some output data.

Key Characteristics:

- **Scheduled:** It processes data at scheduled intervals, such as daily, weekly, or monthly.
- **High throughput:** Since large volumes of data are processed together, it achieves high throughput.
- **Latency:** There is inherent latency because data is processed after being accumulated over a period.
- **Consistency:** It ensures that the entire dataset is processed consistently.

Example Use Cases:

- Generating end-of-day reports.
- Processing payroll at the end of the month.
- Performing large-scale ETL (Extract, Transform, Load) tasks.

Batch Processing Workflow

1. Data Collection:

Data is collected over time and stored in a buffer, file system, database, or data warehouse. This phase can last for hours, days, or even months, depending on the business requirement.

2. Pre-processing:

Before the batch is processed, the system may perform a series of preparatory steps such as data validation, cleaning, filtering, or aggregating.

3. Batch Execution:

Once the data is ready, it is processed as a single unit.

This can involve:

- Running computations over the data.
- Performing ETL (Extract, Transform, Load) operations.
- Aggregating, summarizing, or transforming the data. Batch jobs are usually executed by a batch processing system or job scheduler that triggers the execution at scheduled intervals.

4. Post-processing:

After the execution, the processed data is typically written back to the database, storage, or sent to another system for further use. Reporting or analytics tasks can be triggered in this step.

5. Job Completion:

Finally, the system marks the batch as complete and prepares for the next scheduled run.

Code Example:

```
import pandas as pd

# Sample batch data (could be loaded from a file or database)
batch_data = [
    {"name": "Alice", "transactions": 100},
    {"name": "Bob", "transactions": 150},
    {"name": "Charlie", "transactions": 200},
]

# Convert data to a DataFrame for processing
df = pd.DataFrame(batch_data)

# Perform a batch aggregation to calculate total transactions
total_transactions = df['transactions'].sum()

print(f"Total Transactions Processed in Batch: {total_transactions}")
```

In this simple example, all the data is collected and processed in one go, typical of batch processing.

Challenges in Batch Processing:

- **Latency:** The time it takes to collect data can be a bottleneck in situations where timely insights are required.
- **Storage:** Batch systems require large amounts of storage to collect and hold data before processing. Storing massive datasets over time can be costly.
- **Resource Spikes:** Batch processing jobs often consume significant system resources when they run, causing spikes in CPU, memory, and disk usage. This can slow down other system operations, especially during peak hours.

- **Failure and Error Handling:** If an error occurs during a batch process, the entire batch may need to be reprocessed, leading to inefficiency and delays.

Frameworks and Tools

1. Apache Hadoop

Hadoop is one of the most popular batch processing frameworks. It uses the **MapReduce** paradigm to split large datasets across a distributed cluster, process the data in parallel, and then aggregate the results.

2. Apache Spark

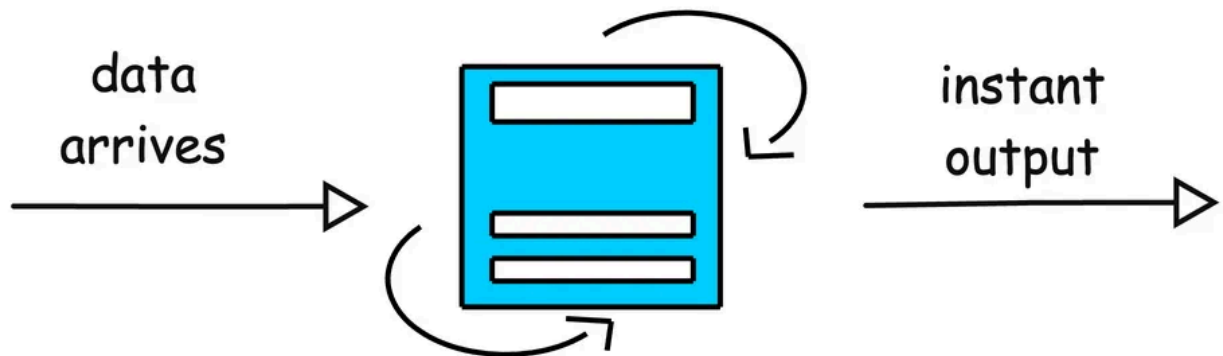
Spark is a distributed computing system that supports batch processing through its **RDD (Resilient Distributed Dataset)** architecture. Spark is more efficient than Hadoop for certain workloads due to its in-memory processing capabilities.

3. AWS Batch

AWS Batch allows developers to easily and efficiently run batch jobs on the cloud. It automatically provisions resources based on job requirements and scales resources up or down as needed.

2. Stream Processing

Stream Processing



blog.algomaster.io

Stream processing is a more recent approach that has gained popularity with the rise of **real-time data** and the need for immediate insights.

It involves processing data in **real time** or **near real time** as it arrives.

Key Characteristics:

- **Real-time processing:** Data is processed as soon as it is received.
- **Low latency:** Stream processing systems are designed to provide low-latency responses, often within milliseconds or seconds.
- **Event-driven:** Processing is triggered by events, making it suitable for real-time applications.
- **Infinite data streams:** Stream processing systems work on continuous flows of data, as opposed to finite datasets in batch processing.

Example Use Cases:

- Monitoring sensor data in IoT systems.
- Detecting fraud in financial transactions in real-time.
- Real-time analytics for online user activity.

Stream Processing Workflow

1. Data Ingestion:

The first step is the ingestion of a continuous flow of data from one or more sources. This data could come from:

- Message brokers like **Apache Kafka** or **AWS Kinesis**.
- IoT devices that generate sensor data.
- Log streams from web servers.
- Financial systems generating transaction records.

2. Processing/Transformation:

Once data is ingested, it is processed as it arrives. Stream processing involves operations such as:

- **Filtering:** Removing unnecessary or irrelevant data.
- **Aggregation:** Summarizing data in real time (e.g., calculating running totals).
- **Windowing:** Grouping events within a specific time window (e.g., calculating metrics over the last 10 minutes).
- **Enrichment:** Joining the stream with external data sources to add more context to the event.

3. State Management:

Many stream processing tasks require the system to maintain state (e.g., aggregating transactions per user).

Managing state in a distributed, real-time environment is complex, but modern frameworks provide fault-tolerant state management to ensure consistency.

4. Output:

The processed data can be used to trigger immediate actions or be sent to other systems like databases, dashboards, or external APIs.

Common destinations include:

- Databases for real-time analytics.
- Alerts or notifications for abnormal events (e.g., fraud detection).
- Updates to a real-time dashboard for visualization.

Code Example:

```
from collections import deque
import time

# Simulating a stream of data (transactions)
transaction_stream = deque([100, 150, 200, 250, 300])

# Function to process each transaction in the stream as it arrives
def process_stream():
    while transaction_stream:
        transaction = transaction_stream.popleft()
        print(f"Processing Transaction: {transaction}")
        time.sleep(1) # Simulating real-time processing delay

# Start processing the stream
process_stream()
```

In this example, transactions are processed one by one as they come in, typical of stream processing where the data is constantly flowing.

Challenges in Stream Processing:

- **Complexity:** The system must handle data continuously, requiring real-time monitoring, scaling, and fault tolerance mechanisms.
- **Data consistency:** Since data is processed in real time, maintaining consistency, especially in distributed systems, can be difficult.

- **Error Handling:** In stream processing, errors need to be managed in real time. Recovering from failures while maintaining accurate results can be challenging, especially in distributed systems where state is maintained across multiple nodes.

3. Frameworks and Tools

1. Apache Kafka

Kafka is one of the most popular distributed messaging systems used for ingesting data streams. It provides high-throughput, fault-tolerant, and scalable message processing. Kafka acts as a buffer between data producers and consumers in stream processing.

2. Apache Flink

Flink is a stream processing framework known for low-latency, high-throughput processing. It supports stateful computations and allows users to build robust real-time data pipelines.

3. AWS Kinesis

Kinesis is Amazon's managed stream processing service. It enables real-time data ingestion and processing at scale, providing seamless integration with other AWS services like Lambda, S3, and Redshift.

4. Batch vs Stream processing: A tabular comparison

Feature	Batch Processing	Stream Processing
Data Processing	Processes a large volume of data at once.	Processes data as it arrives, record by record.
Latency	High latency, as processing happens after data collection.	Low latency, providing near real-time insights.
Throughput	Can handle vast amounts of data at once.	Optimized for real-time but might handle less data volume at a given time.
Use Case	Ideal for historical analysis or large-scale data transformations.	Best for real-time analytics, monitoring, and alerts.
Complexity	Relatively simpler to implement with predefined datasets.	More complex, requires handling continuous streams.
Data Scope	Operates on a finite set of data.	Operates on potentially infinite streams of data.
Error Handling	Errors can be identified and corrected before execution.	Requires real-time handling of errors and failures.
Resource Usage	Resource-intensive during processing, idle otherwise.	Continuous use of resources.
Cost	Cost-effective for large volumes of data.	More expensive due to continuous processing.

5. Choosing the Right Approach

When deciding between batch processing and stream processing, consider the following factors:

- **Data volume:** If you're dealing with large volumes of data, batch processing might be the better choice.

- **Real-time requirements:** If your application requires immediate insights or actions based on incoming data, stream processing is the way to go.
- **Complexity:** If your processing tasks require complex algorithms and data transformations, batch processing might be more suitable.
- **Data nature:** Is your data finite and predictable in size, or an unbounded, ongoing flow? Batch processing is better suited for the former, stream processing for the latter.

Hybrid Approach: Micro-Batch Processing

Some systems like **Apache Spark Streaming** employ a hybrid approach known as **micro-batch processing**.

This method bridges the gap between traditional batch and stream processing by processing small chunks of data over short intervals.

This allows for near real-time processing with the simplicity of batch processing.

Thank you for reading!

If you found it valuable, hit a like ❤️ and consider subscribing for more such content every week.

If you have any questions or suggestions, leave a comment.

This post is public so feel free to share it.

P.S. If you're enjoying this newsletter and want to get even more value, consider becoming a [paid subscriber](#).

As a paid subscriber, you'll unlock all **premium articles** and gain full access to all [premium courses](#) on [algomaster.io](#).

There are [group discounts](#), [gift options](#), and [referral bonuses](#) available.

Checkout my [Youtube channel](#) for more in-depth content.

Follow me on [LinkedIn](#), [X](#) and [Medium](#) to stay updated.

Checkout my [GitHub repositories](#) for free interview preparation resources.

I hope you have a lovely day!

See you soon,

Ashish



135 Likes · 8 Restacks

← Previous

Next

Discussion about this post

Comments Restacks



Write a comment...



Jiten Jiten Oct 19, 2024

♥ Liked by Ashish Pratap Singh

Thanks for the detailed explanation about Batch & Stream Processing !

♥ LIKE (1) 💬 REPLY

© 2025 Ashish Pratap Singh · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture